Układy Cyfrowe i Systemy Wbudowane	
Kierunek *Informatyka techniczna*	**Termin** *Wtorek NP, 11:10*
Imię, nazwisko, numer albumu *Karolina Obiedzińska 273034, Bartłomiej Krupa 272982*	**Data** *21 sierpnia 2025*

Sprawozdanie 4

1 Zadanie 1

1.1 Opis zadania

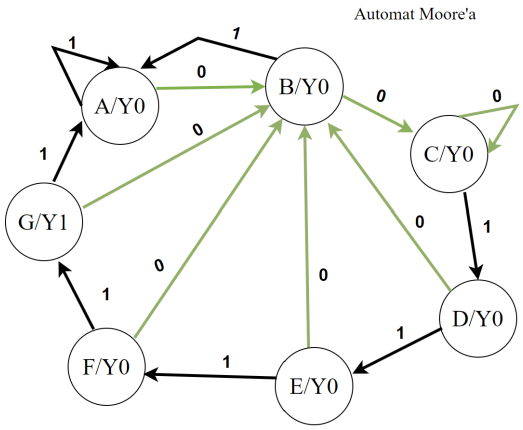
Do wykonania jest detektor sekwencji 6-bitowych zrealizowany jako automat Moore'a.
Nasza sekwencja to: **001111**. Automat powinien spełniać następujące założenia:

- **Wejście**: X,
- **Wyjście**: Y,
- **Dodatkowe sygnały**: CLK, $Reset$.

Zegar systemowy powinien pracować z częstotliwością $F_{CLK} = 40\,\text{MHz}$, a wartość sygnału wejściowego X ma być ustawiana $T = 10\,\text{ns}$ przed zboczem zegara.

1.2 Graf sekwencji

Rysunek nr 1 pokazuje automat Moore'a, z sześcioma stanami (A/Y0 – G/Y1). Wyjście jest przypisane do każdego stanu (Y0 = 0 lub Y1 = 1), a przejścia między stanami zależą wyłącznie od sygnałów wejściowych.



Rysunek 1: Automat Moore'a

1.3 Program

1.3.1 Behavioral

Na rysunku nr 2, 3, 4 widzimy implementację wszystkich trzech funkcji dla automatu Moore'a w języku VHDL.
Funkcja 1: Realizuje logikę synchroniczną opartą na zboczu narastającym sygnału zegarowego (CLK). Proces ten obsługuje reset (RST) oraz sygnał włączenia (CE), aktualizując stan state na wartość *nextstate* w odpowiednich warunkach.
Funkcja 2: Implementuje logikę przejść pomiędzy stanami automatu. Zależnie od aktualnego stanu (*state*) oraz wartości wejścia (X), ustalany jest kolejny stan (*nextstate*). Każdy przypadek dla poszczególnych stanów (A, B, C, itd.) jest realizowany w ramach struktury case.
Funkcja 3: Proces, odpowiedzialny za dekodowanie sygnałów wyjściowych (Q, Y) na podstawie aktualnego stanu automatu (*state*). Dla każdego stanu przypisywane są konkretne wartości wyjść, które są zdefiniowane warunkami if.

```
SYNC_PROC : process(CLK)
begin
   if rising_edge(CLK) then
      if RST = '1' then
         state <= A;
      elsif CE = '1' then
         state <= next_state;
      end if;
   end if;
end process SYNC_PROC;
```

Rysunek 2: Funkcja 1

```
NEXT_STATE_DECODE: process(state, X)
begin
   next_state <= state;
   case state is
      when A =>
         if X = '0' then
            next_state <= B;

         end if;

      when B =>
         if X = '0' then
            next_state <= C;
         else
            next_state <= A;
         end if;

      when C =>
         if X = '1' then
            next_state <= D;
         end if;

      when D =>
         if X = '1' then
            next_state <= E;
         else
            next_state <= B;
         end if;

      when E =>
         if X = '1' then
            next_state <= F;
         else
            next_state <= B;
```

Rysunek 3: Funkcja 2

```
OUTPUT_DECODE: process (state)
begin
   if state = G then
      Q <= "110";
      Y <= '1';
   else
   Y <= '0';
   if state = A then
      Q <= "000";
   elsif state = B then
      Q <= "001";
   elsif state = C then
      Q <= "010";
   elsif state = D then
      Q <= "011";
   elsif state = E then
      Q <= "100";
   elsif state = F then
      Q <= "101";
   end if;
   end if;


end process OUTPUT_DECODE;
```

Rysunek 4: Funkcja 3

1.3.2 TestBench

Na rysunku nr 5 przedstawiono fragment kodu pliku testBench, w którym zainicjalizowano przykładowy wektor wejściowy example oraz skonfigurowano okres sygnału zegarowego, ustawiając wartość CLK_{period} na 25 ns.
Na rysunku nr 6 zaprezentowano fragment kodu pliku testBench, w którym pokazujemy wprowadzenie przykładowego inputu.

```
--Inputs
signal X : std_logic := '0';
signal CE : std_logic := '1';
signal CLK : std_logic := '0';
signal RST : std_logic := '0';
signal example: std_logic_vector ( 20 downto 0) :="01100111100100111 0110";
--Outputs

signal Y : std_logic;

-- Clock period definitions
constant CLK_period : time := 25 ns;
```

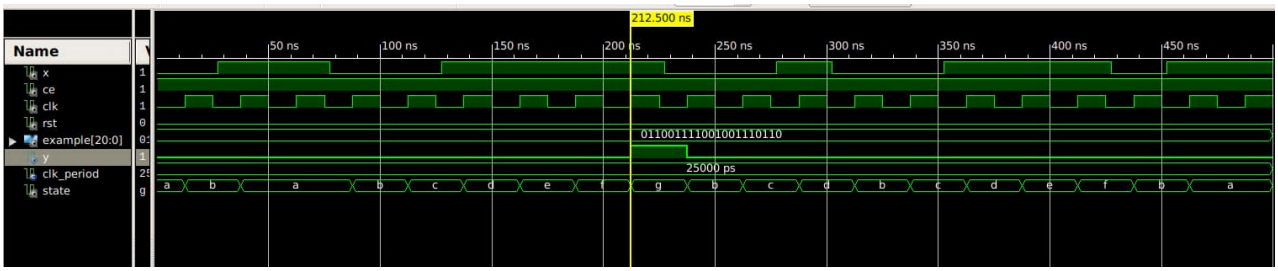
Rysunek 5: testBench

```
-- Stimulus process
stim_proc: process
begin

   for i in 20 downto 0 loop

      wait for 2.5 ns;
      X <= example(i);
      wait for 22.5 ns;

   end loop;

   wait;
end process;
```

Rysunek 6: testBench - sprawdzenie sekwencji

1.3.3 Symulacja czasowa

Na rysunku nr 7 znajduje się symulacja behawioralna dla automatu Moore'a.



Rysunek 7: Symulacja czasowa dla automatu Moore'a

1.4 Obliczenia

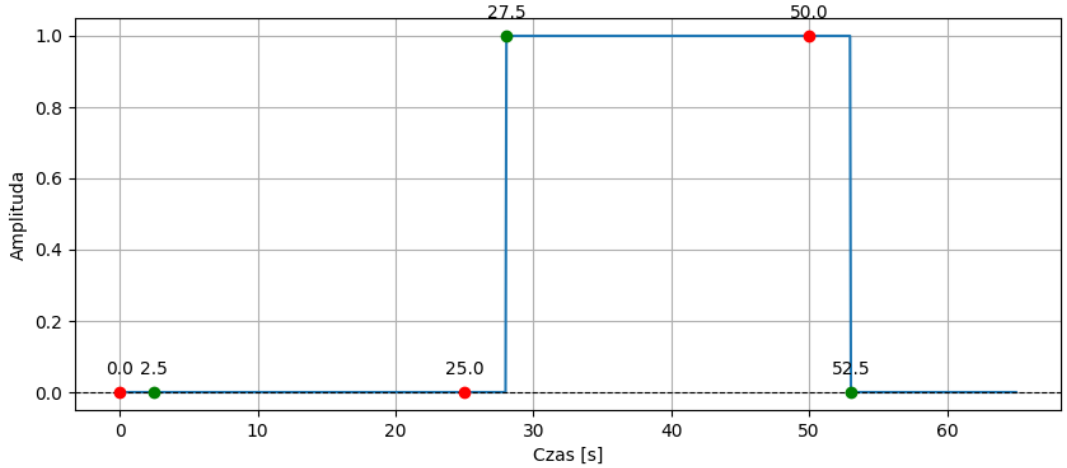
1.4.1 Częstotliwość zegara

Częstotliwość zegara: $f = 40MHz \implies f = 4 \cdot 10^7 Hz$

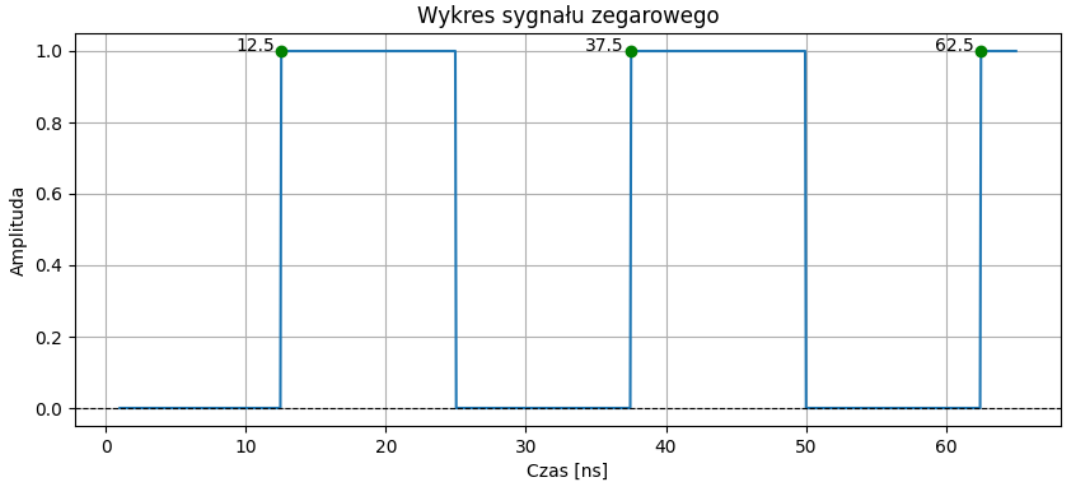
Ze wzoru: $T = \frac{1}{f}$ podstawiając dane: $T = \frac{1}{40 \cdot 10^6 \frac{1}{s}} = \frac{1}{40}\mu s = 25\ ns$

$T = 25\ ns$

1.4.2 Ustawienie sygnału wejściowego X



Rysunek 8: Wykres sygnału wejściowego dla sekwencji 010. Zielone punkty wskazują momenty, w których sygnał wejściowy powinien osiągnąć właściwy stan. Natomiast czerwone punkty oznaczają początki i końce okresów sygnału zegarowego.



Rysunek 9: Wykres czasowy zegara. Zielonymi punktami zaznaczone jest, kiedy sygnał zegara przechodzi z niskiego do wysokiego stanu.

Omówienie: Okres zegara wynosi 25 ns, co znaczy, że co 12.5 ns zmienia swój stan na zmianę z niskiego na wysoki i odwrotnie. Oznacza to, że dla sygnału wejściowego pierwsze ustawienie stanu musi być wykonane 2.5 ns po starcie zegara, a po ustawieniu czekać kolejne 22.5 ns na stan wysoki sygnału zegarowego (tak, aby suma czekania przed ustawieniem stanu i po ustawieniu była równa okresowi sygnału zegarowego).

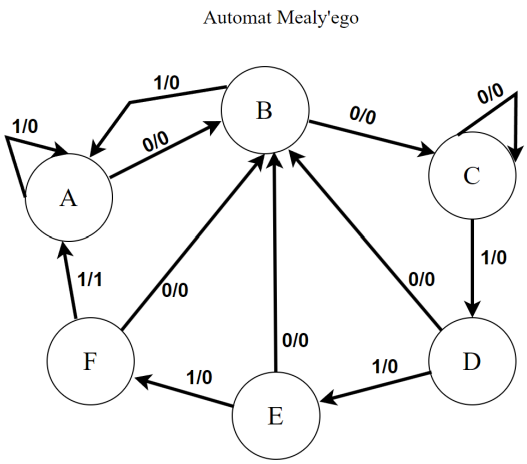
2 Zadanie 2

2.1 Opis zadania

Wykonanie detektora sekwencji w formie automatu Mealy'ego, analogicznego do rozwiązania przedstawionego w poprzednim zadaniu.

2.2 Graf sekwencji

Rysunek nr 10 przedstawia automat Mealy'ego z sześcioma stanami (A–F). Przejścia między stanami są oznaczone parami wejście/wyjście, co oznacza, że wyjście zależy zarówno od aktualnego stanu, jak i od sygnału wejściowego



Rysunek 10: Automat Mealy'ego

2.3 Program

Plik VHDL był analogiczny do tego z poprzedniego zadania, z tą różnicą, że nie uwzględniono dodatkowego stanu. Zmodyfikowana została jedynie funkcja odpowiedzialna za dekodowanie wyjścia, którą przedstawiamy poniżej. TestBench był identyczny w obu przypadkach.

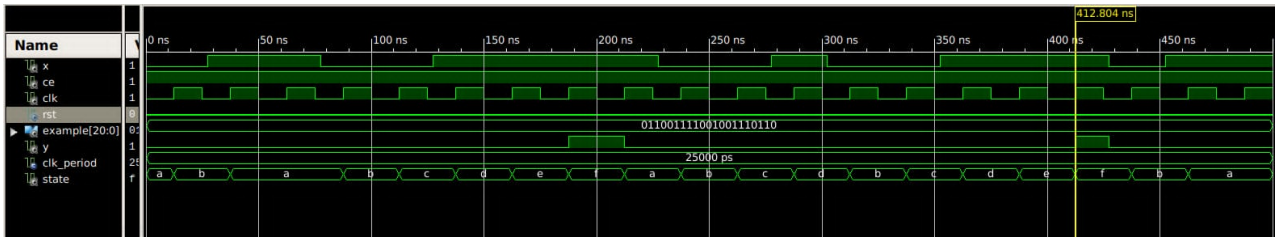
Na rysunku nr 11 widzimy fragment, którym różnił się program automatu Mealy'ego od automatu Moore'a.

```
OUTPUT_DECODE: process (state, X)
begin
   if state = F and X = '1' then
      Q <= "101";
      Y <= '1';
   else
      Y <= '0';
   end if;
   if state = A then
      Q <= "000";
   elsif state = B then
      Q <= "001";
   elsif state = C then
      Q <= "010";
   elsif state = D then
      Q <= "011";
   elsif state = E then
      Q <= "100";
   end if;


end process OUTPUT_DECODE;
```

Rysunek 11: Funkcja 3 dla automatu Mealy'ego

2.3.1 Symulacja czasowa

Na rysunku nr 12 znajduje się symulacja behawioralna dla automatu Mealy'ego.



Rysunek 12: Symulacja czasowa dla automatu Mealy'ego

3 Wnioski

3.1 Porównanie automatu Moore'a i Mealy'ego

- Automat Moore'a: Wyjście zmienia się tylko w chwilach, gdy maszyna przechodzi z jednego stanu do drugiego.
- Automat Mealy'ego: W obrębie tego samego stanu, wyjście może się zmieniać w odpowiedzi na różne wejścia.

3.2 Rozpoznanie stanu wyjściowego w automacie Mealy'ego

W automacie Mealy'ego stan przejściowy to stan, w którym wyjście $y = 1$ powoduje zmianę stanu w odpowiedzi na określone wejście. Natomiast stan zgłaszający to stan, w którym wyjście $y = 1$ jest generowane bez zmiany stanu, a automat pozostaje w tym stanie przez co najmniej jeden cykl.

3.3 Podsumowanie

Automat Moore'a jest bardziej przewidywalny, prostszy do projektowania, jednak wymaga większej liczby stanów niż automat Mealy'ego.
Zrealizowano symulację oraz implementację automatu na sprzęcie.